.UBAfiuba
FACULTAD DE INGENIERÍA

(TA050) Sistemas Distribuidos I

Curso 01 - Roca

Documento de arquitectura: Wikipedia Article Analysis

13 de Octubre de 2026

Pedro Santiago Gillen
112333

Melanie Belén García Lapegna
111848

Martina Riccardi
111859

Índice

1. Alcance **4**

1.1. Fuera del Alcance 4

2. Vista de Escenarios **5**

2.1. Escenario consulta 1 6

2.2. Escenario consulta 2 6

2.3. Escenario consulta 3 6

2.4. Escenario consulta 4 7

2.5. Escenario consulta 5 8

3. Visión General de la Arquitectura **8**

3.1. Etapa de procesamiento 9

4. Objetivos y Restricciones **10**

4.1. Objetivos 10

4.2. Restricciones 11

5. Vista Lógica **11**

6. Vista de Procesos **14**

6.1. Explicación interna de procesos 14

6.1.1. Procesos Consultas 1 y 2 15

6.1.2. Procesos Consultas 3 y 4 16

6.1.3. Procesos Consulta 5 17

6.1.4. Flujo completo 18

6.2. Dinámica de Procesos 18

6.2.1. Una instancia a muchas a través de una cola 18

6.2.2. Muchas instancias a muchas a través de un exchange 21

6.2.3. Muchas instancias a muchas a través de una cola 22

7. Vista de desarrollo **23**

7.1. Diagramas de paquete 23

7.1.1. General 23

7.1.2. Paquete Common 24
7.1.3. Paquete Workers 25
7.2. Capas 25

8. Vista de Despliegue **27**
8.1. Controllers y flujos de datos 27
8.2. Nodos y contenedores 29

9. Calidad **29**
9.1. Apéndice A: Acrónimos y abreviaturas 32
9.2. Apéndice B: Definiciones 32

1. Alcance

El sistema distribuido analiza artículos de Wikipedia y resuelve cinco consultas:

1. **Q1:** artículos en inglés hasta 2020.
2. **Q2:** artículos escritos hasta 2020 que tienen una sección de biografía.
3. **Q3:** palabras únicas compartidas entre artículos en inglés y en francés.
4. **Q4:** las 20 palabras más utilizadas en los abstracts.
5. **Q5:** artículos escritos desde 2021 cuyo abstract supera el largo promedio de los escritos hasta 2020.

Las consultas se resuelven en paralelo y el detalle de cada una de estas está en la sección de escenarios.

Los datos a utilizar provienen del dataset Wikipedia Structured Contents, publicado por Wikimedia Enterprise en Kaggle. Contiene los artículos de las ediciones en inglés y francés de Wikipedia

El sistema ingiere los siguientes datos:

- `abstract`: resumen del artículo.
- `date_modified`: fecha de última modificación.
- `image`: imagen principal.
- `in_language`: idioma del artículo.
- `name`: título del artículo.
- `sections`: secciones del artículo.
- `url`: dirección del artículo.

El envío de los mismos se realiza mediante un proceso *Cliente* provisto, el cual lee estos campos y los envía al sistema.

En las consultas que permitan despachar resultados parciales, estos se enviarán a medida que se obtengan (como en Q1), evitando así la necesidad de almacenarlos.

1.1. Fuera del Alcance

A continuación, se detallarán algunas cuestiones que quedaron por fuera del alcance del sistema:

- Tolerancia a Fallos en el sistema
- Almacenamiento de las consultas
- GUI para el cliente

2. Vista de Escenarios

En esta sección se detallan los casos de uso más importantes del sistema, enfocándose en los distintos flujos de ejecución que se llevan a cabo para procesar los datos y responder las consultas del cliente.

Estos escenarios pueden ser representados de manera sencilla por el siguiente diagrama de casos de uso:

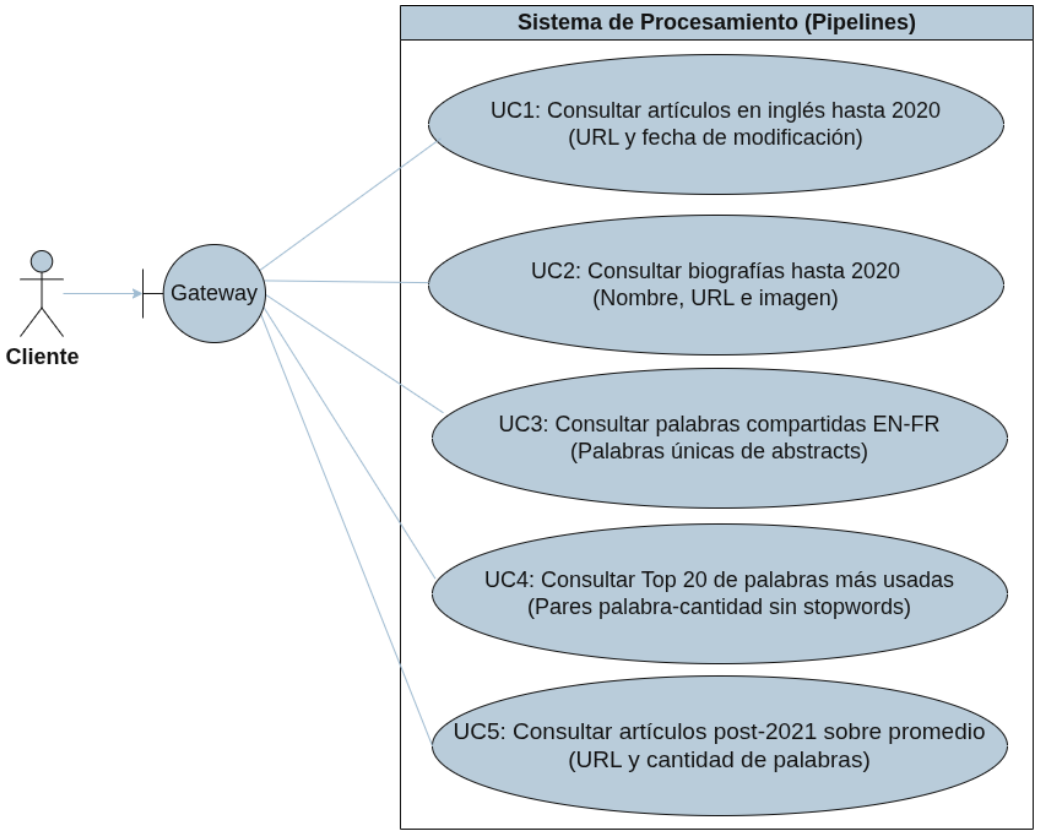


Figura 1: Casos de uso

Todos los escenarios comparten el mismo inicio y el mismo final:

- El **Cliente** envía al **Gateway** cada artículo con los campos `abstract`, `date_modified`, `image`, `in_language`, `name`, `sections` y `url`.
- El **Gateway** le agrega un `client_id` a cada mensaje, para poder atender a varios clientes a la vez, y lo publica en las colas de entrada de los distintos pipelines.
- Todos los resultados llegan a la **Results Queue**, de donde el **Gateway** los toma y se los devuelve al cliente correspondiente según su `client_id`.

2.1. Escenario consulta 1

El objetivo de la misma es retornarle al cliente la **url y fecha de modificación** de los artículos escritos en inglés hasta el 2020 inclusive.

Un ejemplo de flujo es:

1. El **Cliente** envía los artículos al **Gateway**, que agrega el `client_id` y los publica en la cola que alimenta al **Filtro año**.
2. El **Filtro año** descarta los artículos con `date_modified` posterior a 2020. Los restantes se reenvían con `client_id`, `url`, `in_language` y `date_modified`.
3. El **Filtro idioma** conserva solo los artículos con `in_language` igual a inglés y reenvía `client_id`, `url` y `date_modified`.
4. El resultado se publica en la **Results Queue**.
5. El **Gateway** lo toma, identifica al cliente mediante `client_id` y le devuelve la url y la fecha de modificación.

2.2. Escenario consulta 2

El objetivo de la misma es retornarle al cliente el **nombre, url e imagen (bytes)** de los artículos escritos hasta el 2020 inclusive que incluyan "biography" o "biographie" entre sus secciones.

Un ejemplo de flujo es:

1. El **Cliente** envía los artículos al **Gateway**, que agrega el `client_id` y los publica en la cola que alimenta al **Filtro año**.
2. El **Filtro año** descarta los artículos posteriores a 2020 y reenvía `client_id`, `name`, `url`, `image` y `sections`.
3. El **Filtro biografía** conserva solo los artículos cuyas `sections` incluyen "biography" o "biographie" y reenvía `client_id`, `name`, `url` e `image` (bytes).
4. El resultado se publica en la **Results Queue**.
5. El **Gateway** lo toma, identifica al cliente mediante `client_id` y le devuelve nombre, url e imagen.

2.3. Escenario consulta 3

El objetivo de la misma es retornarle al cliente el **listado de palabras únicas compartidas** entre cualquier artículo escrito en inglés y cualquier artículo escrito en francés.

Un ejemplo de flujo es:

1. El **Cliente** envía los artículos al **Gateway**, que agrega el `client_id` y los publica en la cola que alimenta al **Tokenizer**, junto con `abstract`, `in_language` y `url`.
2. El **Tokenizer** filtra las stopwords del `abstract` y aplica un `set` para quedarse con las palabras únicas de ese artículo. Envía al pipeline de las consultas 3 y 4 los mensajes `client_id`, `[word, abstract_appearances, in_language]`.
3. El **Hasher** toma cada palabra y la publica en un exchange con **afinidad por palabra**. Así, todas las apariciones de una misma palabra, sin importar el artículo o el idioma, llegan a la misma instancia de **Word counter**.
4. Cada instancia de **Word counter** guarda la cantidad recibida de cada palabra, teniendo en cuenta el idioma de origen.
5. Cuando una palabra tiene apariciones en ambos idiomas, el **Word counter** la envía inmediatamente (una sola vez) a la **Results Queue** como resultado de la consulta 3, con `client_id` y `words`.
6. El **Gateway** toma los resultados, identifica al cliente mediante `client_id` y le devuelve las palabras compartidas.

2.4. Escenario consulta 4

El objetivo de la misma es retornarle al cliente los **pares (palabra, cantidad) del top 20 de palabras más utilizadas** entre todos los abstracts. Dentro de un mismo abstract cada palabra se cuenta una sola vez y las stopwords se excluyen.

Un ejemplo de flujo es:

1. El **Cliente** envía los artículos al **Gateway**, que agrega el `client_id` y los publica en la cola que alimenta al **Tokenizer**.
2. El **Tokenizer** filtra las stopwords y aplica un `set` sobre el `abstract`, de modo que cada palabra se cuente una sola vez por artículo. Envía `client_id`, `[word, abstract_appearances, in_language]` al pipeline de las consultas 3 y 4.
3. El **Hasher** publica cada palabra en un exchange con afinidad por palabra, de modo que una misma palabra siempre llega a la misma instancia de **Word counter**.
4. Cada instancia de **Word counter** acumula la cantidad de abstracts en los que aparece cada palabra que le corresponde.
5. Al recibir el **EOF** del cliente, cada instancia ya tiene sus conteos completos y calcula su **top 20 parcial**. Lo envía como `client_id`, `top_20_parcial` a un exchange con **afinidad por cliente**.
6. El **Join Top 20** recibe los tops parciales de todas las instancias de Word counter para ese cliente y calcula el **top 20 global**.

7. El resultado (`client_id`, `top_20`) se publica en la **Results Queue**.
8. El **Gateway** lo toma, identifica al cliente mediante `client_id` y le devuelve los pares (palabra, cantidad).

2.5. Escenario consulta 5

El objetivo de la misma es retornarle al cliente la **url y cantidad de palabras de abstract** de los artículos escritos a partir del 2021, cuya cantidad de palabras de abstract sea superior al promedio de todos los artículos escritos hasta el 2020 inclusive. Se aplica el mismo criterio para contabilizar palabras que en la consulta anterior (sin stopwords y sin repeticiones).

Un ejemplo de flujo es:

1. El **Cliente** envía los artículos al **Gateway**, que agrega el `client_id` y los publica en la cola que alimenta al **Tokenizer**.
2. El **Tokenizer** filtra las stopwords y calcula `len(filtered_abstract)`. Según el año del artículo, lo envía por una de dos colas:
   - Artículos de 2021 en adelante: `client_id`, `len(filtered_abstract)`, `url`, hacia el **Average filter**.
   - Artículos de 2020 o anteriores: `client_id`, `len(filtered_abstract)`, `url`, hacia el **Sum**.
3. El **Sum** acumula, por cliente, la longitud total de los abstracts (`total_len`) y la cantidad de artículos (`total_abstracts`). Los envía a un exchange con **afinidad por cliente**.
4. El **Sum Join** combina las sumas parciales de todas las instancias de Sum para ese cliente y calcula el **promedio** (`total_len` / `total_abstracts`). Lo publica por **broadcast** (`client_id`, `promedio`) para que lo reciban todas las instancias del Average filter.
5. El **Average filter** compara la cantidad de palabras de cada artículo de 2021 en adelante contra el promedio. Mientras el promedio no llegó, los artículos se guardan en archivos para no perderse.
6. Los artículos que superan el promedio se publican en la **Results Queue** como `client_id`, `url`, `len(filtered_abstract)`.
7. El **Gateway** lo toma, identifica al cliente mediante `client_id` y le devuelve la url y la cantidad de palabras.

3. Visión General de la Arquitectura

El sistema sigue un **estilo de procesamiento de mensajes mediante colas de mensajería**. Esto permite reducir cuellos de botella mediante el escalado independiente de cada etapa. Las diferentes consultas se resuelven mediante Pipes-Filters (Q1 y Q2) y Scatter-Gather (Q3, Q4 y Q5).

La principal desventaja de este modelo es la alta cantidad de colas y mensajes que se terminan enviando dentro del sistema. Esto podría generar una latencia notable o una sobrecarga en el MOM.

Para reducir la sobrecarga de comunicación, se identifican los nodos con menor carga de procesamiento y se consolidan sus responsabilidades en un único nodo lógico. Esto permite reducir la cantidad de colas y mensajes intercambiados a través del MOM.

3.1. Etapa de procesamiento

Se muestran las distintas etapas de procesamiento detectadas y cómo se es el flujo entre ellas. Esto se realiza a partir de un diagrama DAG, el cual fue utilizado como guía para el diseño final del sistema.

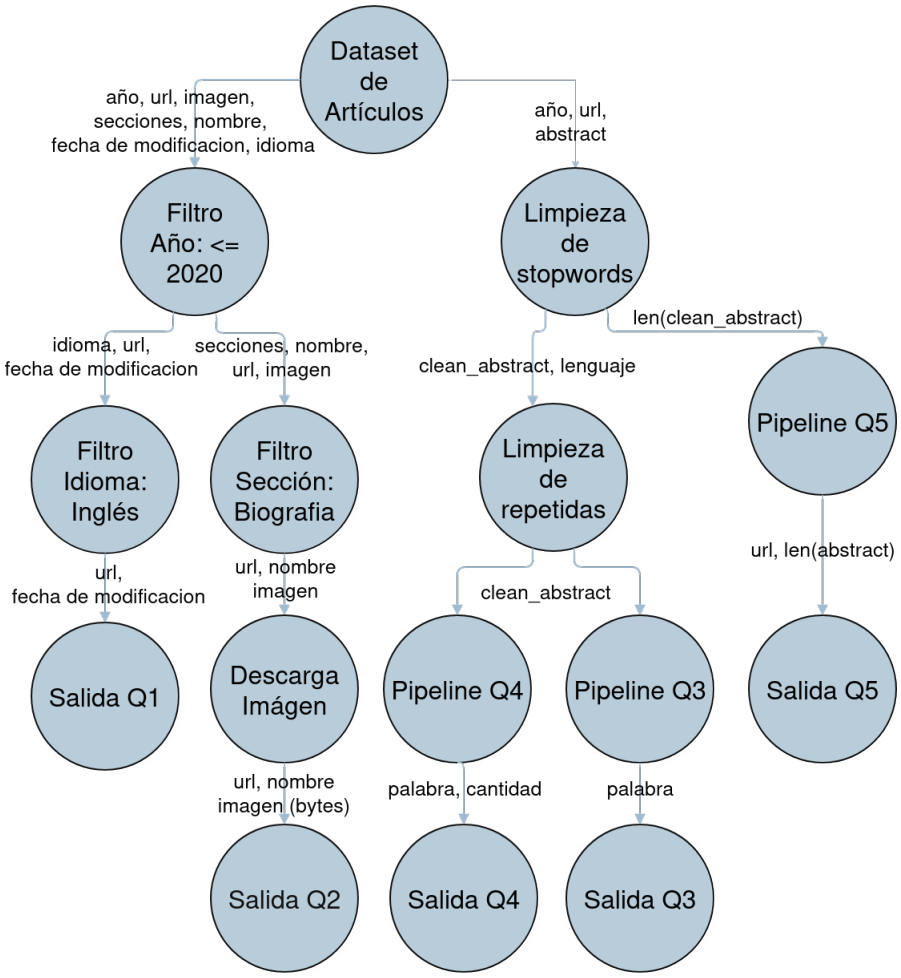


Figura 2: Directed Acyclic Graph

El filtrado temprano por año se aprovecha para Q1 y Q2 y la limpieza de los abstractos también se reutiliza en las consultas Q3, Q4 y Q5.

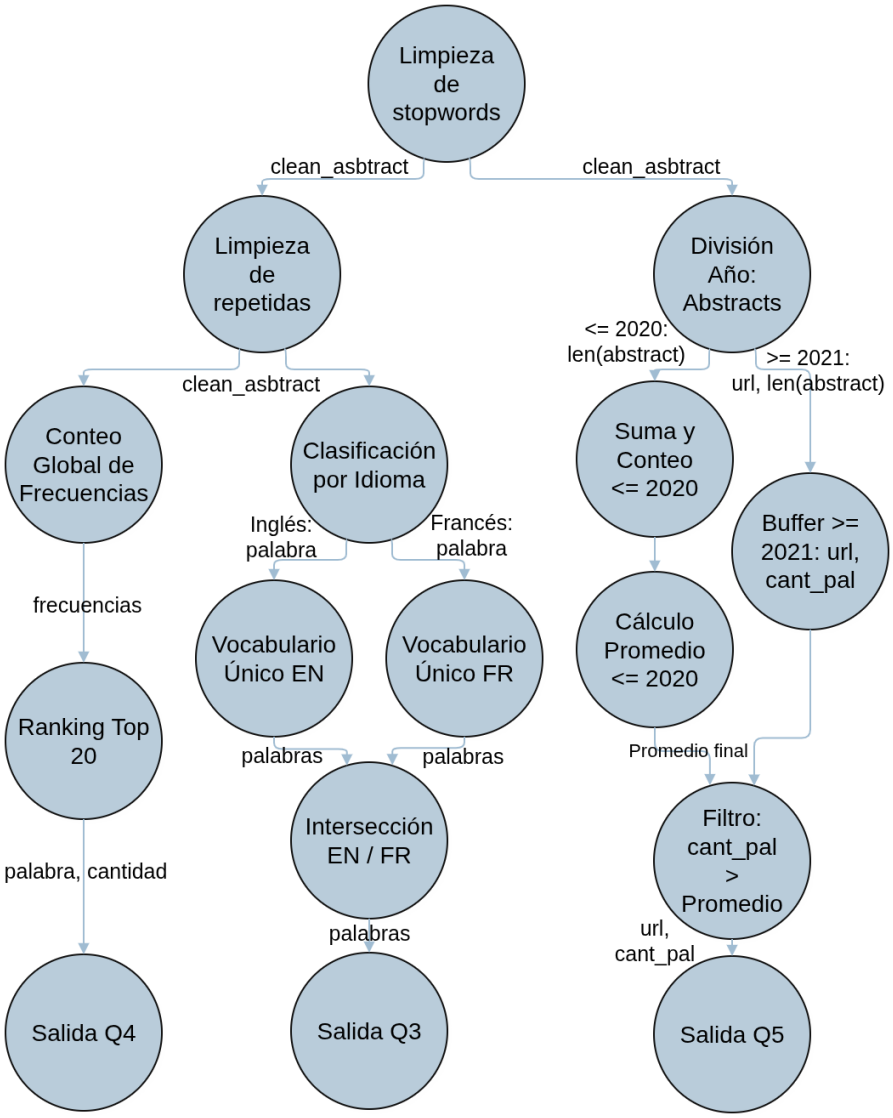


Figura 3: DAG: Q3 - Q5

En el mismo se notan etapas de filtrado y los procesamientos que requieren de un Scatter-Gather (tener un conteo de todas las palabras para tener un top 20 global, por ejemplo).

No todas las etapas mencionadas son nodos en el sistema debido a optimizaciones encontradas.

4. Objetivos y Restricciones

En cuanto a los objetivos y las restricciones que se tuvieron en cuenta o se descubrieron al momento de diseñar el sistema.

4.1. Objetivos

Los objetivos que guian el sistema son:

- **Escalabilidad Horizontal:** Se busca que el sistema pueda escalar añadiendo más nodos de un cierto tipo para reducir cuellos de botellas
- **Rendimiento mediante Filtrado Temprano:** Filtrar y proyectar los datos lo antes posibles permite reducir la carga de comunicación y procesamiento entre los nodos
- **Consistencia de los datos:** Se busca que las consultas sean respondidas de forma correcta y en su totalidad

4.2. Restricciones

Las restricciones detectadas fueron las siguientes:

- **Dependencias temporales en Q5:** Al necesitar procesar todos los datos posteriores a 2020 previo al filtrado, se tienen que acumular los datos de 2021 en adelante. Esto puede generar una sobrecarga en la memoria y disco de los nodos encargados de esta tarea.
- **Nodos con estado:** En los nodos que necesitan mantener un estado (Q3, Q4 y Q5) se podría llegar a sobrecargar la memoria de los mismos y se tendría que empezar a bajar a disco, lo que podría realentizar el procesamiento final

5. Vista Lógica

El sistema se compone de múltiples entidades que cooperan para realizar el procesamiento distribuido. La mayoría se agrupa bajo la abstracción **Worker**, que encapsula la lógica de procesamiento. Para ello, se define la clase abstracta `BaseWorker`, que centraliza la interfaz y la lógica común, mientras que cada worker específico implementa el procesamiento específico de su etapa.

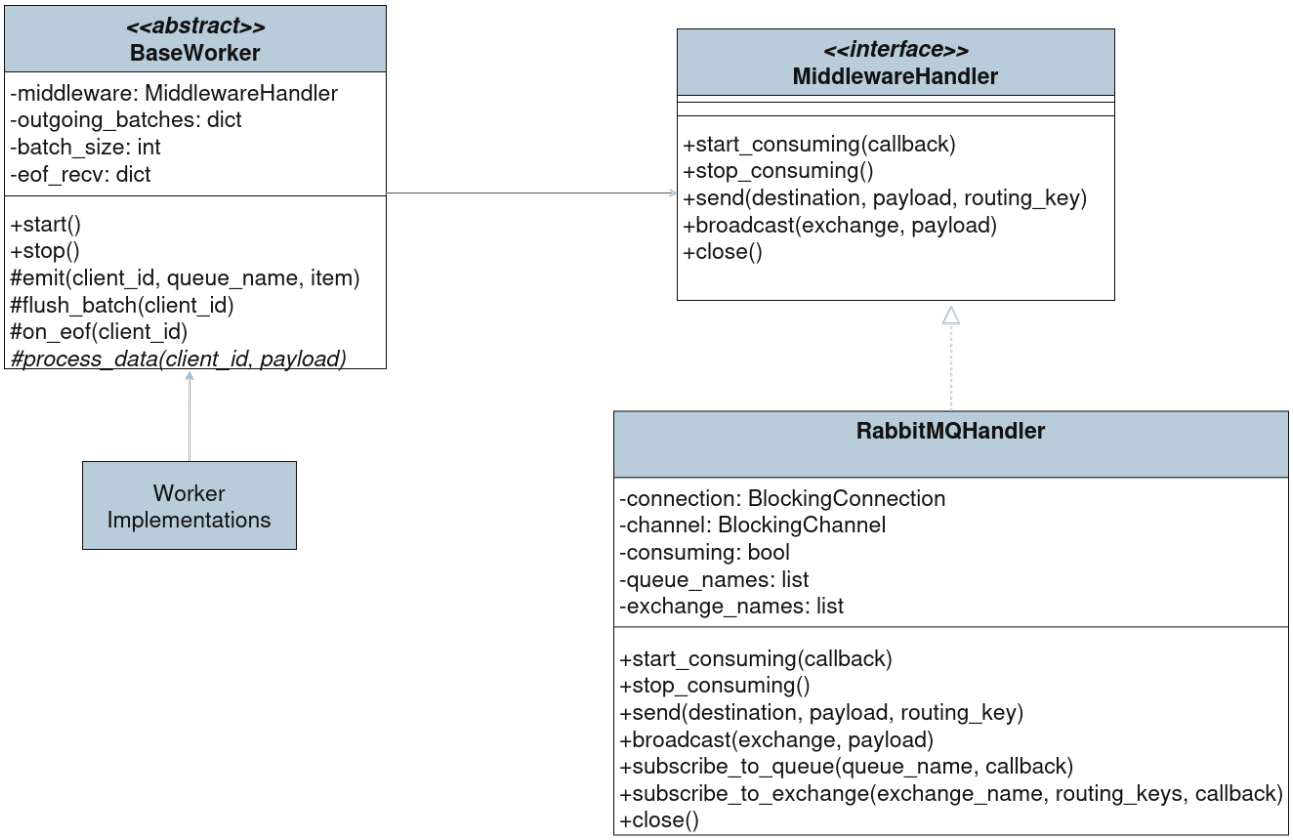


Figura 4: Diagrama de clases

Un BaseWorker implementa:

- Un ciclo de vida para cada proceso con start() y stop()
- Envío en batch con `emit(client_id, item)` que agrega un elemento y `flush_batch(client_id)` que lo envía cuando se llega a `batch_size` o cuando llegue un EOF.
- Recepción con `on_data_message` y `on_control_message` que separan los mensajes de datos.
- Y el fin de datos con `on_eof` y `eof_recv` que van a llevar la cuenta de los EOF que recibe el cliente. `on_barried_or_finished` que se va a invocar cuando se confirma que todas las réplicas termianron de procesar a un cliente.

Lo único que cada uno de los Workers va a tener que implementar es `process_data(client_id, payload)`, que contiene la lógica del proceso en sí.

De esta clase abstracta BaseWorker van a heredar los diez workers del sistema: **Year filter, Biography filter, Language filter, Average filter, Tokenizer, Word counter, Hasher, Sum, Sum Join y Top 20 Join.**

Tener esta clase abstracta evita tener código repetido y además si en algún momento se quiere incorporar a un nuevo worker bastará con definirle únicamente el process_data.

Por otra parte, se tienen módulos que son utilzados por todos o la gran mayoría de procesos del sistema. Entre ellos, se encuentran el módulo de protocolo, encargado de serializar y deserializar los datos intercambiados, y el módulo de middleware, que proporciona una interfaz para interactuar con RabbitMQ.

Tambien, se define la clase `EofCoordinator`, que encapsula los mecanismos de sincronización necesarios para coordinar las distintas etapas del pipeline. Esta entidad distingue entre los workers que consumen de una cola y los que consumen de un exchange, y adapta a cada caso la forma de detectar que todos los mensajes de un cliente ya fueron procesados.

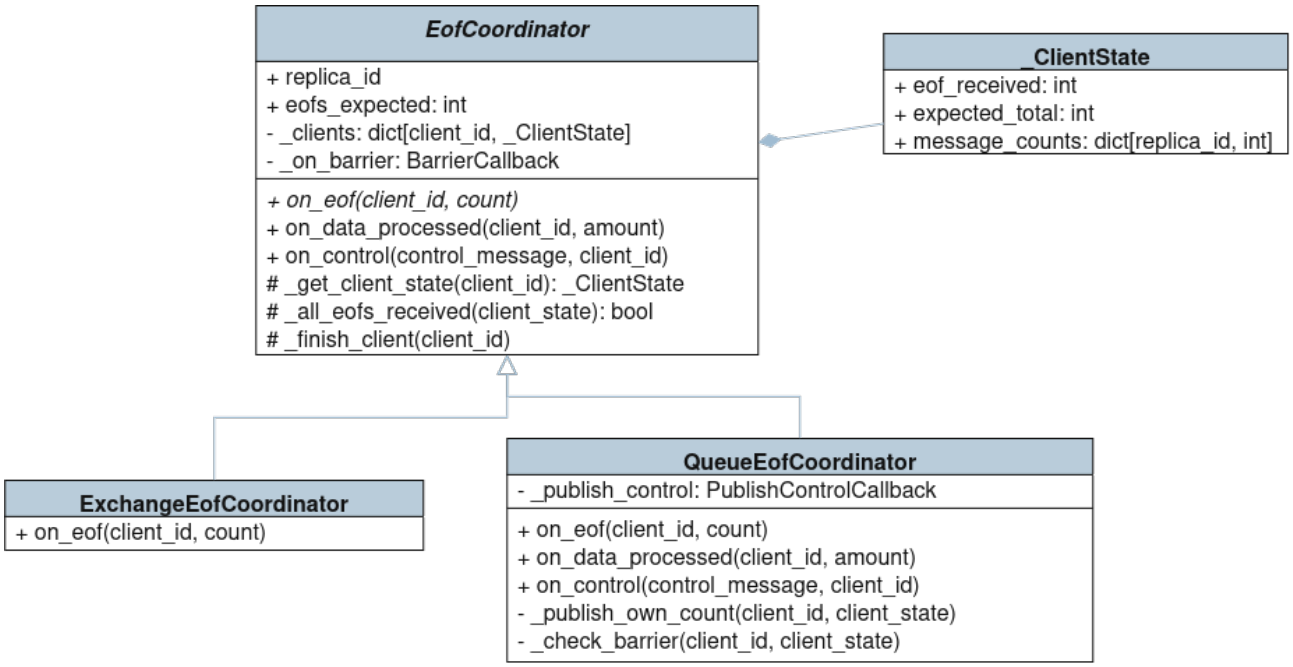


Figura 5: Diagrama de clases del EofCoordinator

Un EofCoordinator implementa:

- Manejo de la recepción de EOF con `on_eof(client_id, message_count)`.
- Notificación de mensajes procesados por el worker con `on_data_processed(client_id, amount)`.
- Recepción de mensajes de control con `on_control(control_message, client_id)`.

Las funciones principales del coordinador deben implementarse según el medio de comunicación utilizado, es decir, *queue* o *exchange*. Particularmente, en el casos de la queue, cuando las réplicas de un worker consumen de una misma cola, cada mensaje es entregado a una sola réplica. Por lo que, ninguna réplica puede determinar por sí sola si ya se procesaron todos los mensajes de un cliente. Para resolverlo, las réplicas se comunican entre sí mediante un exchange de control.

Finalmente, los componentes cliente y gateway se implementan como módulos independientes, dado que encapsulan responsabilidades específicas y no comparten comportamiento con otros nodos.

6. Vista de Procesos

Se describen en detalle los distintos procesos que conforman al sistema distribuido. Para ello, se presentan diagramas de secuencia y actividad para facilitar la comprensión del comportamiento y rol de los mismos.

6.1. Explicación interna de procesos

La comunicación dentro del sistema (todos los nodos menos el gateway y el cliente) se realiza mediante envío de datos en **batch**, salvo que se indique lo contrario. Siempre que se mencione que se envían datos de un nodo a otro, los mismos son guardados en un buffer para ser enviados cuando sea necesario. Esta decisión optimiza el tráfico de red y permite la escalabilidad del sistema reduciendo la congestión.

El sistema cuenta con un único nodo Gateway, el cual se comunica con el cliente. Este recibe todos los datos y los despacha al resto del sistema para su procesamiento. Además, se encarga de recibir los resultados y reenviarlos al cliente correspondiente.

El diagrama muestra como se comportan estos procesos:

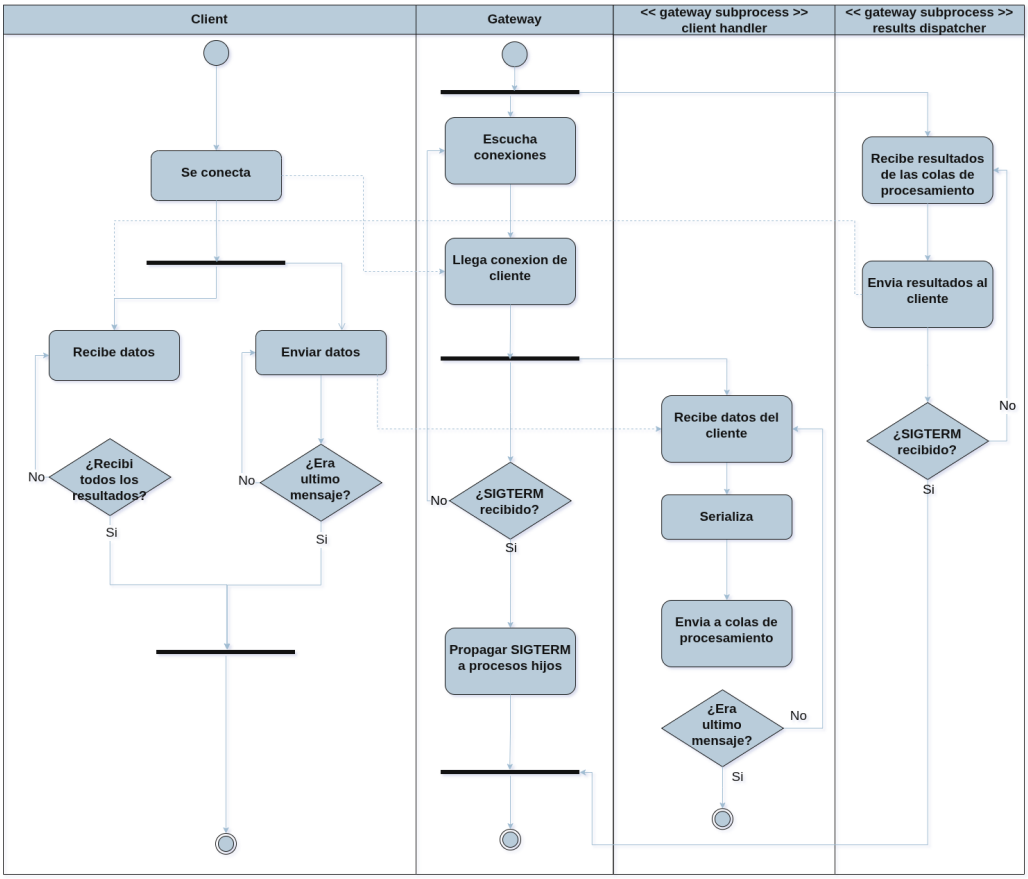


Figura 6: Consulta de ingles por fecha y consulta de palabra por idioma

El cliente al iniciar, crea 2 hilos, uno para el envio de los datos y otro para la recepcion de los

mismos. Al finalizar ambas tareas, termina el proceso cliente.

El Gateway crea 2 procesos, 1 para escuchar conexiones y otro para escuchar los resultados de las consultas. Cuando el primer proceso recibe una conexion, el mismo genera un nuevo proceso para recibir los datos de ese cliente. El segundo proceso se encarga de recibir los datos y redirigirlos al cliente correspondiente. El gateway, cuando recibe la señal de sigterm hace join entre estos 2 procesos, liberando los recursos correctamente.

6.1.1. Procesos Consultas 1 y 2

Para las consultas 1 y 2, se filtran los artículos por año y se proyectan los campos necesarios para cada caso. Luego, la Consulta 1 descarta los artículos cuyos abstracts no están en inglés y envía los resultados, mientras que la Consulta 2 identifica aquellos que contienen «biographie» o «biography» en sus secciones y descarga sus imágenes en un nodo dedicado antes de enviar los resultados. Los nodos se comunican mediante colas, lo que permite distribuir equitativamente el trabajo entre procesos del mismo tipo y aprovechar aquellos que se encuentren menos congestionados.

El diagrama que describe estos procesos es el siguiente:

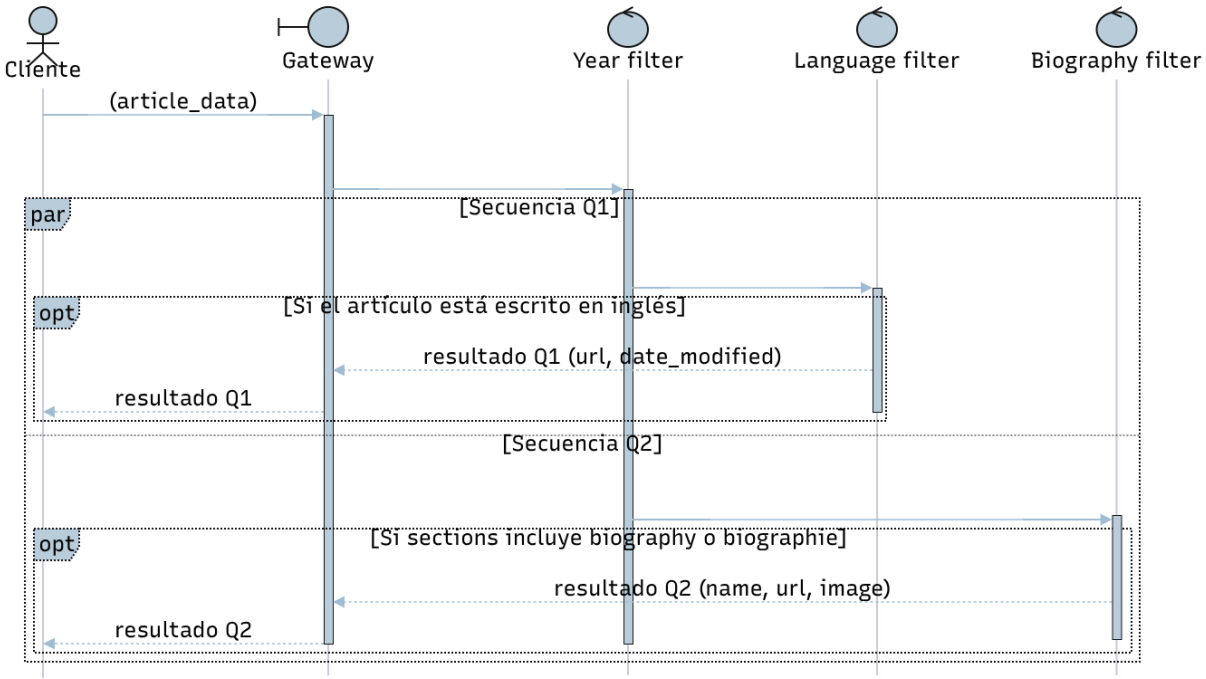


Figura 7: Consulta de ingles por fecha y consulta de palabra por idioma

En el diagrama se observa cómo el cliente envía inicialmente los artículos a procesar al Gateway, el cual los despacha hacia el nodo filtrador por año. A partir de allí, los filtros por idioma y por biografía operan en paralelo, procesando la información y enviando los resultados parciales a medida que se cumplen las condiciones de cada consulta.

6.1.2. Procesos Consultas 3 y 4

Estas consultas comienzan con un proceso que se encarga de la limpieza por stopwords y descarte de palabras repetidas.

Para optimizar la transmisión de datos, las palabras se agrupan en batches. Se mantiene una estructura de diccionarios anidados por cliente que contabiliza las apariciones de cada palabra, diferenciando los artículos en inglés y en francés. Al alcanzar el límite establecido, los datos se empaquetan en una lista con la palabra y ambos contadores, que se envía a la siguiente etapa.

La distribución se realiza mediante exchanges con afinidad por palabra, de modo que todas las apariciones de una misma palabra se procesan en una misma instancia

En la siguiente etapa, los procesos acumulan la frecuencia de cada palabra para ambos idiomas. Si una palabra aparece en artículos de ambos idiomas, cumple con la condición de la Consulta 3 y se envía como resultado.

Una vez recibida la señal de EOF y confirmado que se procesaron todos los mensajes, cada proceso genera un Top 20 parcial con las frecuencias obtenidas. Estos resultados se envían a un nodo Join, encargado de calcular el Top 20 final de cada cliente. Como cada proceso envía un único resultado parcial, esta comunicación se realiza directamente, sin utilizar batches.

El siguiente diagrama muestra como trabajarian estos procesos:

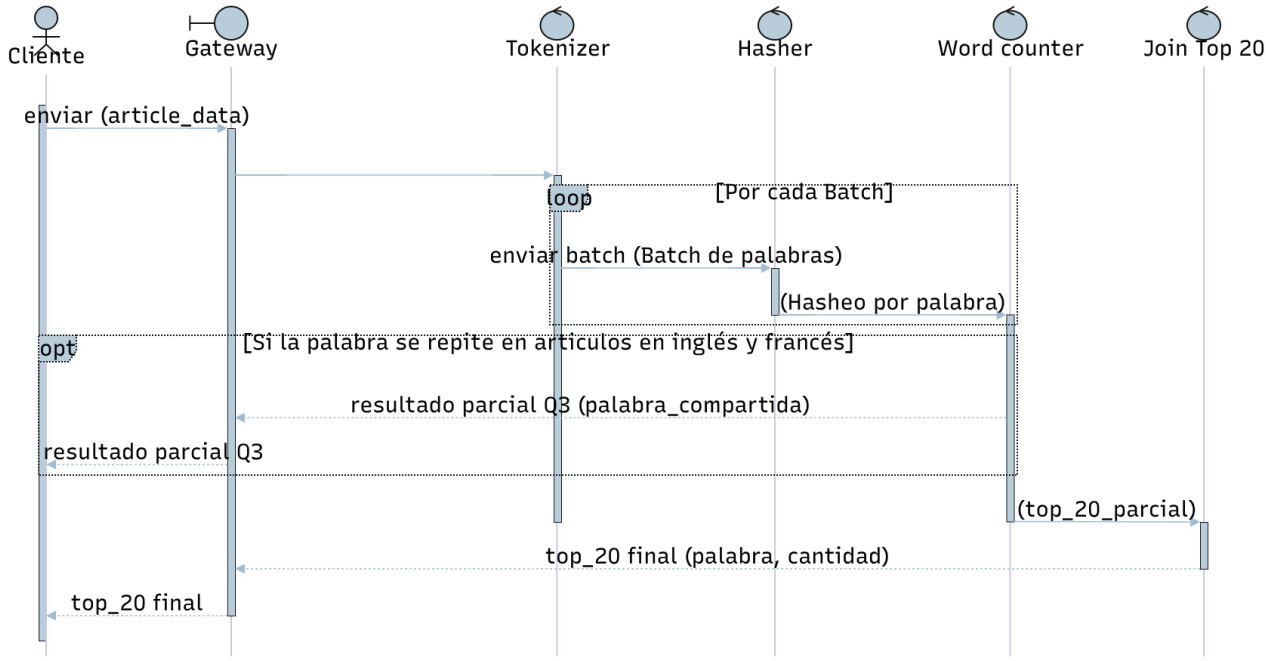


Figura 8: Consulta vocabulario Compartido entre Artículos en Inglés y Francés y consulta top 20 Palabras Más Frecuentes en Abstracts

El diagrama representa el flujo de procesamiento de las consultas 3 y 4, desde la limpieza inicial de los artículos hasta la obtención de los resultados. Se observa cómo los datos se distribuyen entre los diferentes nodos y cómo, una vez finalizado el procesamiento, los resultados parciales se envían al

nodo Join, que calcula el Top 20 final de cada cliente.

6.1.3. Procesos Consulta 5

La primera etapa consta de procesos que suman los largos de los artículos y la cantidad de artículos procesados (luego de haber pasado por el nodo tokenizer). Al recibir el EOF y llegar al consenso de que se procesaron todos los datos del cliente, las sumas son enviadas a la siguiente fase.

Los datos son enviados a un nodo join, el cual se encarga de sumar los resultados parciales para calcular el promedio final. Este promedio se envía al nodo de filtro para asegurarse que todos reciban el promedio.

El nodo de filtro acumula los datos recibidos en disco, teniendo un archivo por cada cliente. Así, no se satura la memoria de los procesos y al recibir el promedio de un cliente, se pueden leer todos los acumulados fácilmente y borrarlos de manera sencilla.

Estos procesos se pueden ver en el diagrama:

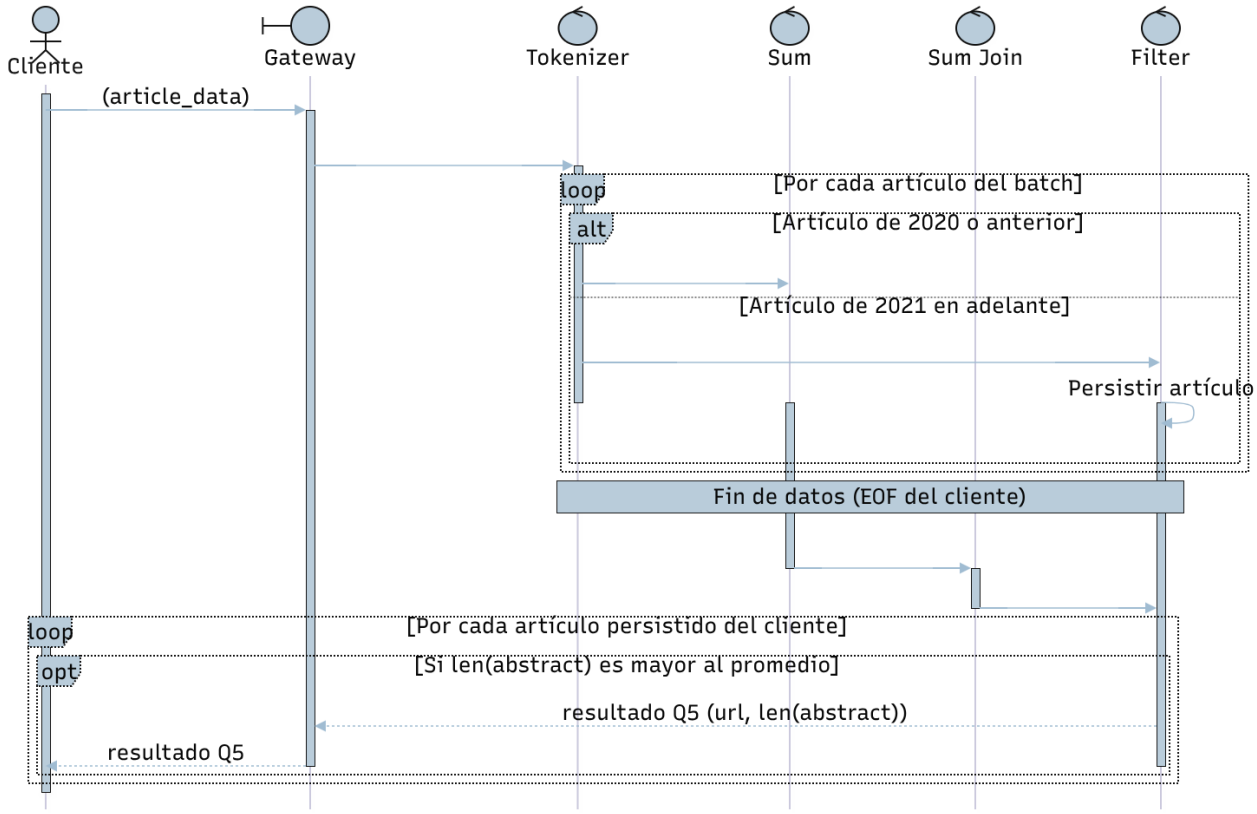


Figura 9: Consulta de abstracts recientes por encima del promedio Hasta 2020

El diagrama ilustra cómo se resuelve la dependencia entre el procesamiento de los artículos publicados hasta 2020 inclusive y el de aquellos publicados a partir de 2021. Mientras los primeros se utilizan para calcular el promedio, los segundos se almacenan temporalmente hasta que este se encuentra disponible. Una vez recibido el promedio, los nodos filtradores procesan los artículos almacenados y envían al Gateway aquellos que cumplen con la condición de la consulta.

6.1.4. Flujo completo

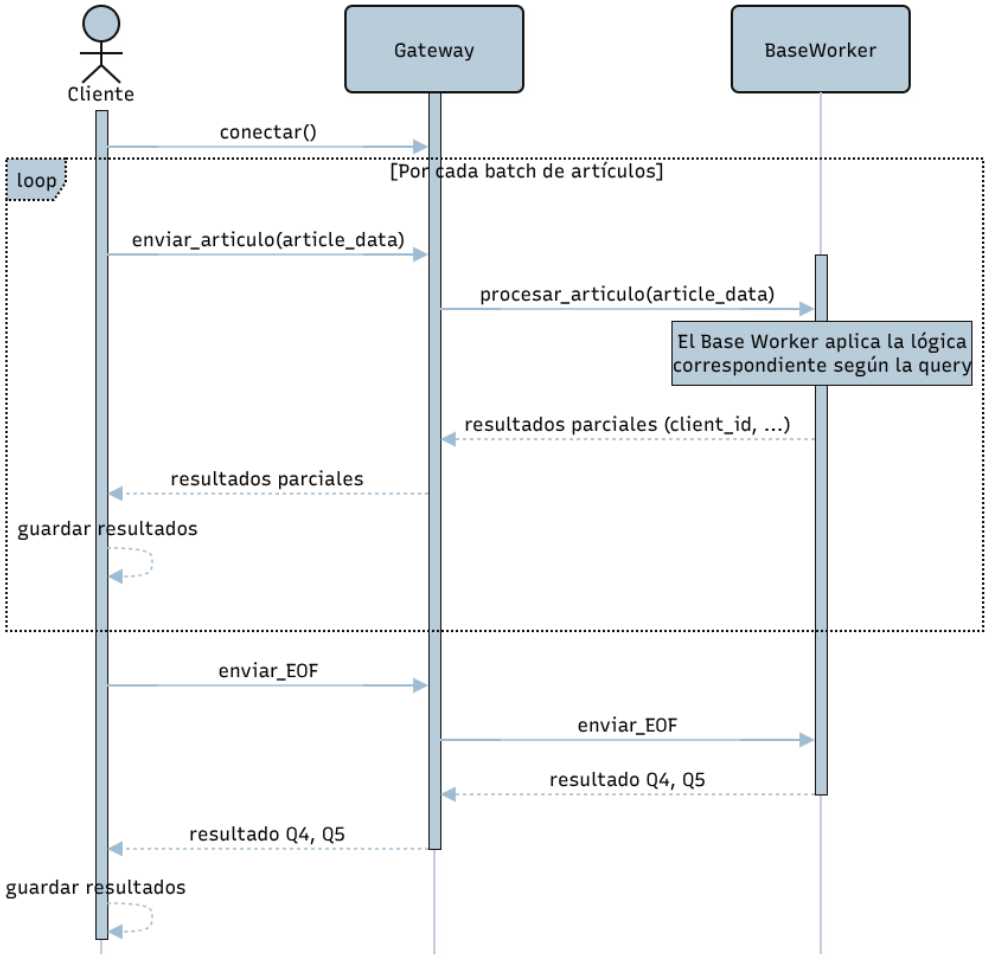


Figura 10: Secuencia completa de consultas

El diagrama modela el flujo completo del sistema estructurado en dos fases temporales delimitadas por el mensaje de control EOF, teniendo una fase de streaming donde el Cliente envía artículos al Gateway y se procesan en paralelo las consultas directas (Q1 y Q2 a través de Year filter, y las detecciones en tiempo real de Q3 mediante Tokenizer), y una fase de sincronización posterior al EOF donde se coordinan las reducciones globales que demandan la totalidad del dataset, emitiendo el Top 20 definitivo de Q4 y los artículos resultantes de Q5 hacia el cliente

6.2. Dinámica de Procesos

6.2.1. Una instancia a muchas a través de una cola

A continuación, se presentan los diagramas de actividades que ilustran el mecanismo de sincronización. Este mecanismo garantiza que cada etapa haya completado el procesamiento de los datos de manera correcta y coordinada antes de pasar a la etapa siguiente.

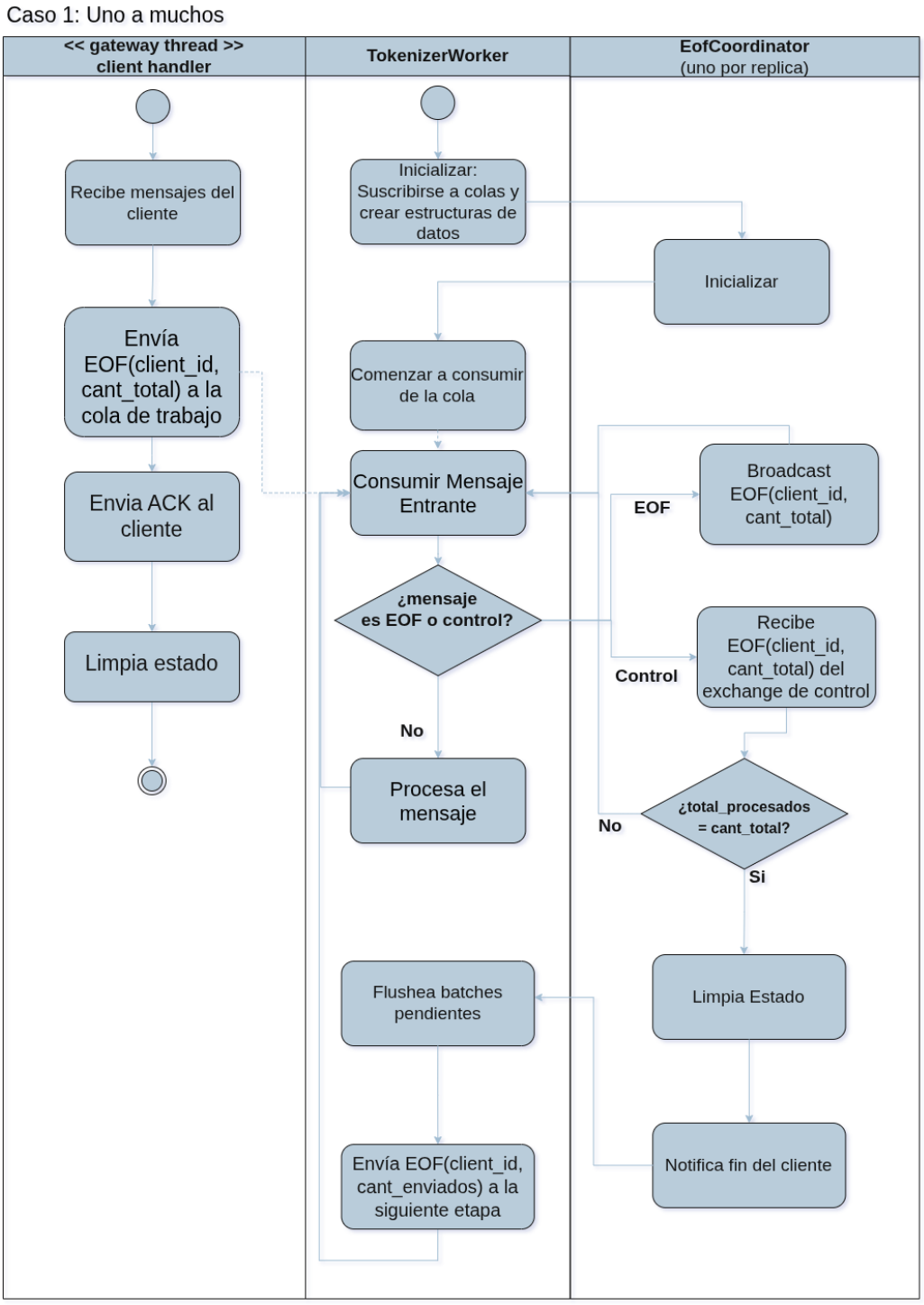


Figura 11: Consolidación de EOF cuando hay 1 emisor del mismo

En este primer diagrama se muestra el caso particular de procesamiento de EOF cuando **la comunicación entre los distintos procesos es a través de una queue**, **el proceso que envía tiene una única instancia** y **el que recibe puede tener múltiples instancias**.

Para simplificar la gestión del EOF, se utiliza la entidad EofCoordinator, encargada de abstraer a los workers de esta lógica según el modelo de comunicación. De este modo, cuando una única instancia recibe el EOF, el coordinador envía un mensaje de broadcast al resto de las instancias (incluyéndose)

para notificar la llegada del EOF junto con la cantidad total de mensajes enviados por la etapa anterior.

Una vez hecho esto, comienza **la secuencia 1**, la cual realizan todas las instancias del proceso.

En el mismo, una vez que llega y se procesa el mensaje de broadcast enviado por el coordinator de una de las réplicas, hace aviso al resto (nuevamente utilizando broadcast) de cuántos mensajes procesó hasta el momento.

Cada vez que una réplica recibe el reporte de mensajes procesados por las demás instancias, verifica si se alcanzó la totalidad de los mensajes enviados. Si el conteo acumulado no coincide, se debe a que alguna de las instancias aún continúa procesando información de ese cliente. Una vez que dichas instancias finalizan y notifican su estado, el conteo alcanza finalmente el total notificado en un inicio por la réplica que consumió el mensaje de EOF de la etapa previa.

A medida que cada una de las réplicas validan que se terminaron de procesar todos los mensajes recibidos para un cliente en particular, entonces, todas envían **el mensaje de EOF** a la próxima fase.

6.2.2. Muchas instancias a muchas a través de un exchange

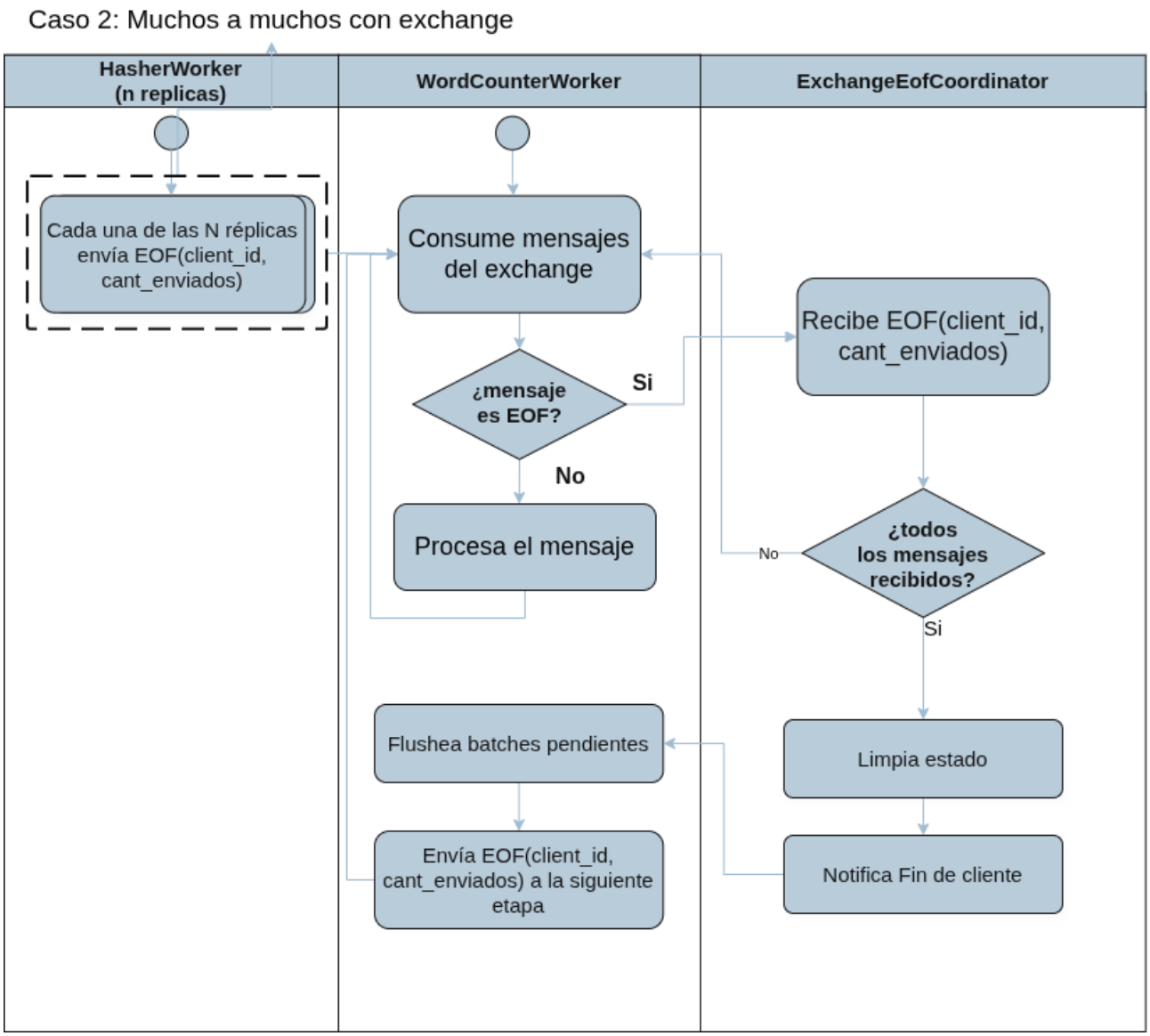


Figura 12: Consolidación de EOF cuando hay N emisores

En este otro diagrama, se analiza el caso particular en el que **un proceso que tiene muchas réplicas envía el mensaje de EOF a otro que también tiene muchas réplicas** y esta comunicación se realiza a través de un **exchange**.

Como muestra el diagrama, todas las instancias del primer proceso **envían un mensaje de EOF** avisando que ya terminaron de enviar todos los mensajes. Entonces, **todas** las réplicas de la nueva etapa, al comunicarse a través de un exchange, reciben el EOF publicado por cada una de las réplicas del proceso anterior.

Esto mencionado, que es a lo que se lo llamó **secuencia 2**, se realiza en loop hasta que la **instancia del nuevo proceso reciba la cantidad de EOF correspondiente a la cantidad de instancias del proceso anterior**.

Cuando esta condición se cumple, se confirma que la réplica terminó de procesar todos los mensajes que le correspondían de las fases previas. A continuación, envía la información procesada necesaria para el siguiente paso, acompañada por su propio mensaje de EOF.

6.2.3. Muchas instancias a muchas a través de una cola

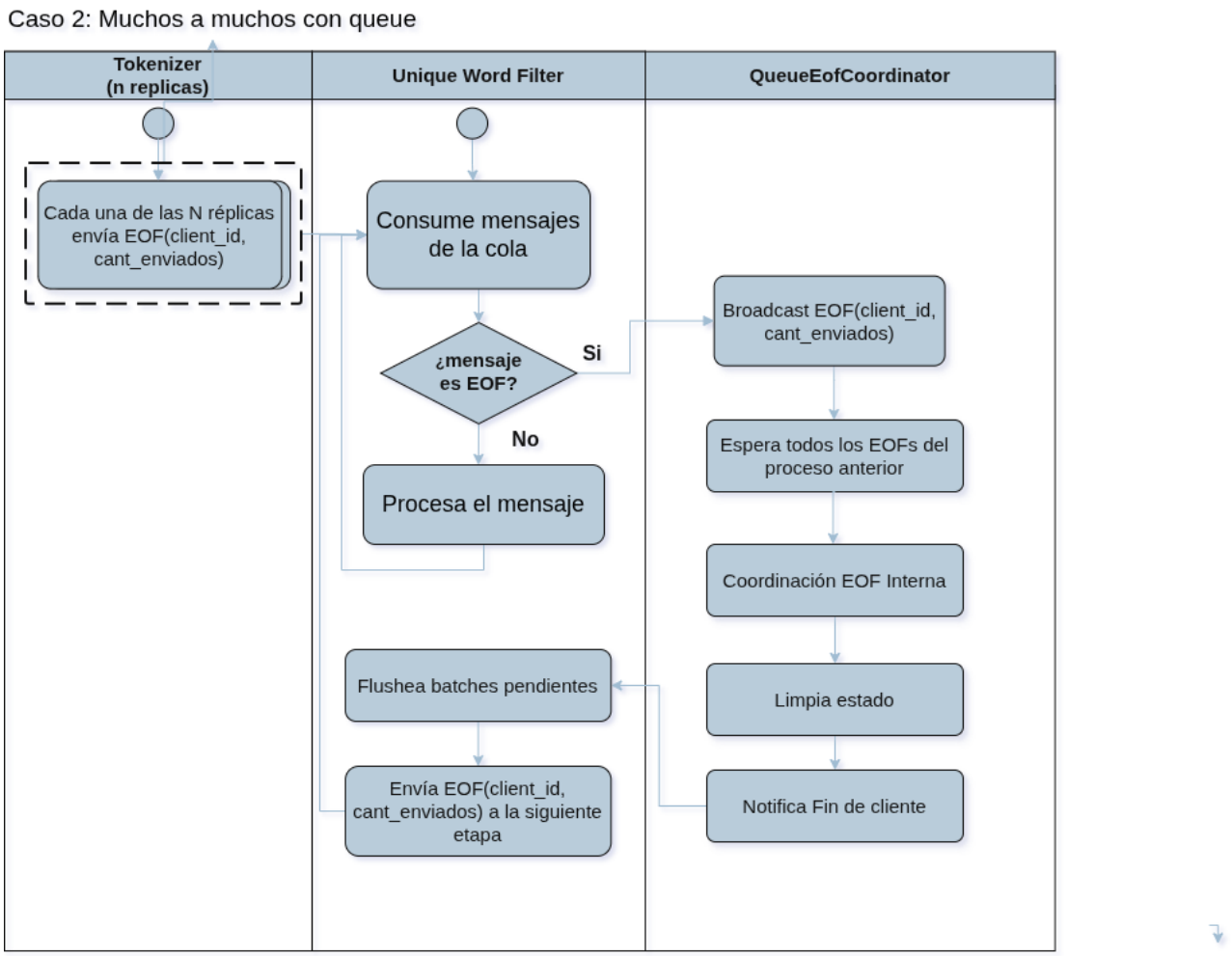


Figura 13: Consolidación de EOF cuando hay N emisores

Este último caso, muestra cómo es la interacción cuando se tiene **muchas instancias para el proceso pasado, muchas instancias para el proceso actual y además la comunicación entre estos procesos es a través de una cola**.

En este caso, se optó por «combinar» la **secuencia 1 y 2** mencionadas para los diagramas anteriores.

Entoces la idea del es que, todas las instancias del proceso anterior envíen el EOF pero **solo una única réplica de la nueva etapa lo va a consumir**. Cuando lo hace, envía un mensaje en broadcast para que comience la etapa menciona por **secuencia 1** en la que básicamente se esperan los N EOFs (siendo N la cantidad de réplicas en el proceso anterior). Una vez que se llega a esa cantidad, se pasa

a la segunda etapa en la cual se sigue el flujo comentado en la **secuencia 2** en la que se espera que todas las instancias del proceso actual **terminen de procesar todos los mensajes enviados por el proceso anterior**.

Una vez que se terminan con ambas etapas, cada una de las instancias actuales proceden a enviar el mensaje de EOF.

7. Vista de desarrollo

7.1. Diagramas de paquete

7.1.1. General

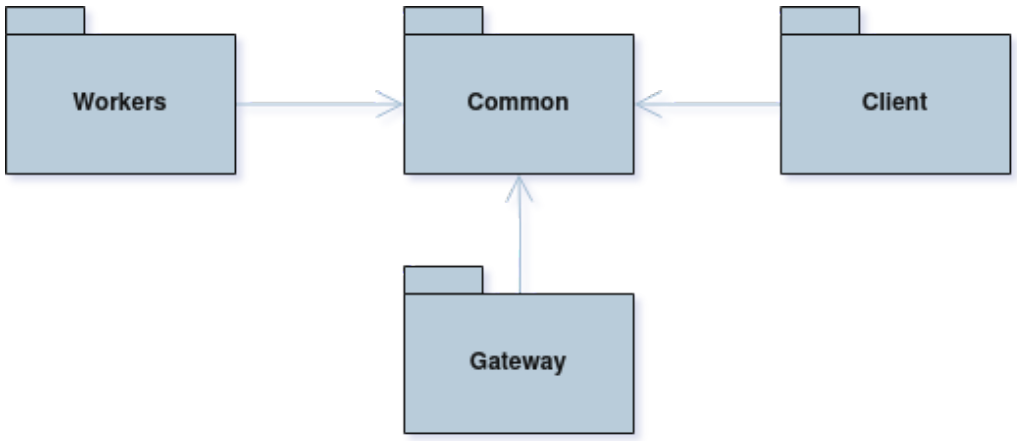


Figura 14: Paquetes principales

En este diagrama se muestran los paquetes principales del sistema y sus dependencias. Los paquetes son cuatro: **Client, Gateway, Workers y Common**. Los tres primeros corresponden a los tres tipos de procesos del sistema y los tres dependen de **Common**, que conecta el código compartido (básicamente es el acceso al middleware y al protocolo de serialización).

Que no haya dependencias entre Client, Gateway y Workers tiene sentido porque estos justamente se comunican a través de Common (colas y exchanges de RabbitMQ) usando el formato de mensaje definido por el mismo paquete. De esta manera, se puede modificar o escalar de manera independiente y Common sería el único punto de acoplamiento entre los mismos.

Además, aparecen dos paquetes **external**:

- pika: es la biblioteca cliente de RabbitMQ que se usa únicamente en Common. Ningún otro paquete lo va a conocer.
- ntlk.corpus: lo usa el proceso de Tokenizer el cual se encuentra dentro del paquete de workers. El mismo es para obtener la lista de stopwords.

7.1.2. Paquete Common

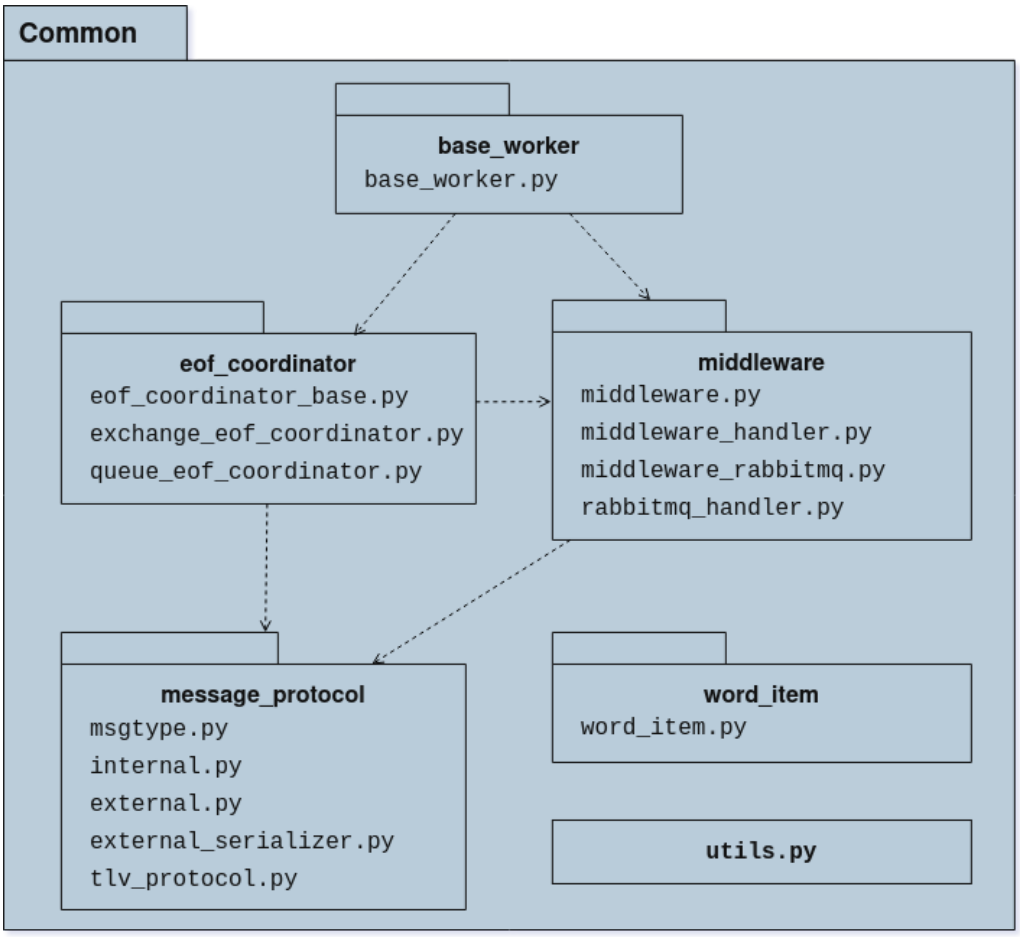


Figura 15: Paquete Common

El paquete Common se divide en dos subpaquetes:

Middleware: Este define la interfaz *MiddlewareHandler* con las operaciones que necesita cualquier proceso para comunicarse. `start_consuming(callback)` y `stop_consuming()` para recibir mensajes, `send(destination, payload, routing_key)` para enviar a una cola o un exchange, `broadcasst(exchanve, payload)` para enviar a todos los suscriptores y `close()` para poder liberar la conexión.

La clase RabbitMQHandler es la interfaz sobre RabbitMQ, guarda la conexión y el canal de pika, junto con otros atributos que se utilizan en la implementación de la misma.

Usar esta interfaz va a permitir que el resto del sistema dependa de esta abstracción y no directamente de RabbitMQ.

Y en cuanto a **Protocol**: se encarga de serializar y deserializar los datos de los mensajes entre los procesos. El mismo expone funciones como `serialize_tlv` y `deserialize_tlv` que conviernen entre la clase Message (que tiene como contenido a una lista a la que se la llama payload) y su representación en bytes con formato TLV (tipo, longitud, valor).

Esto permite dar un único formato de mensje para todo el sistema, y que los mensajes se pueden

agrupan en batchs.

7.1.3. Paquete Workers

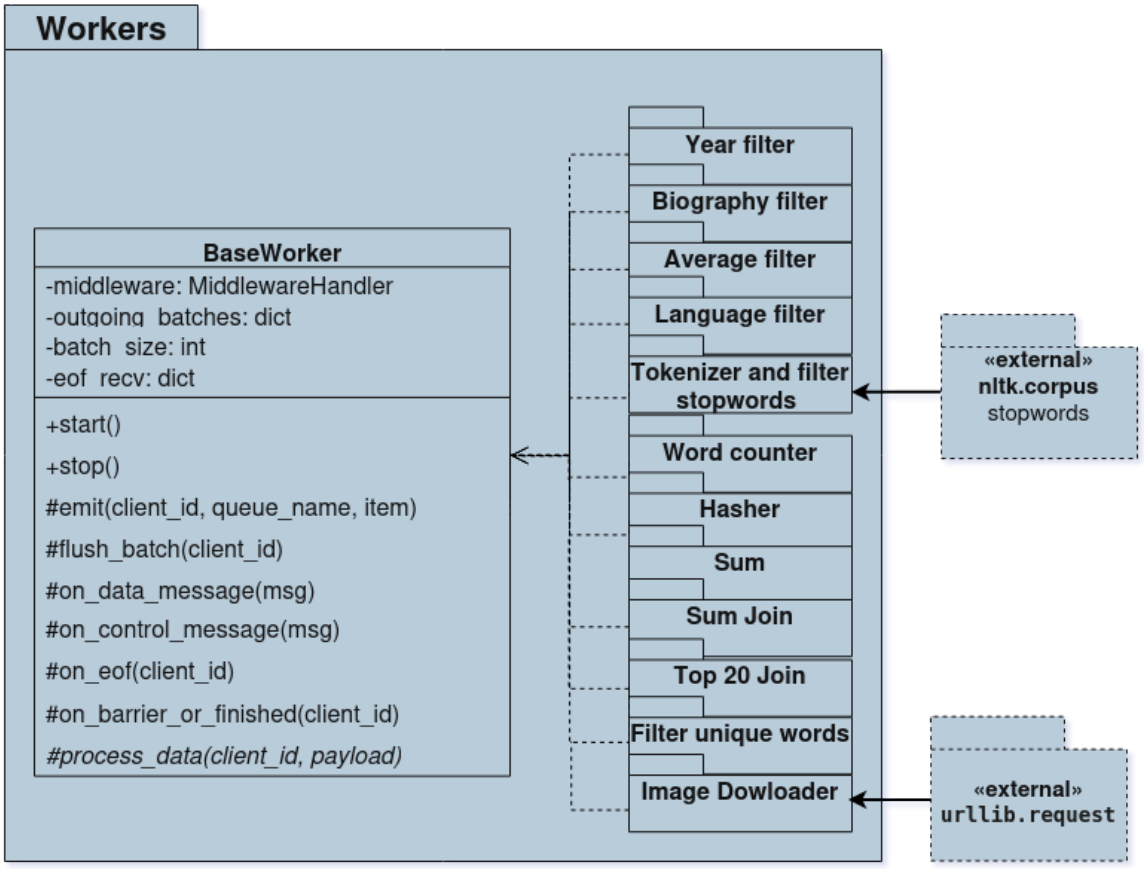


Figura 16: Paquete Workers

El paquete Workers se organiza alrededor de la clase abstracta BaseWorker. Esta clase tiene la lógica y la interfaz compartidas por todos los workers, y recibe un MiddlewareHandler (depende de la abstracción previamente mencionada y no de RabbitMQ).

7.2. Capas

Además de la organización en paquetes, el procesamiento de los datos se puede ver como una división en capas lógica:

1. **Capa de Ingesta y Ruteo:** recibe todos los datos y despacha en dos colas para el procesamiento de los mismos
2. **Capa de Filtros directos:**
   - Filtro por idioma: descarta artículos según la fecha de modificación para concretar la consulta 1.

- Filtro Biografía: descarta artículos según sus secciones para concretar la consulta 2.

3. **Capa de Limpieza:** quita las `stopwords` para las consultas 3 y 4, y se queda además únicamente con las palabras únicas. Para la consulta 5, únicamente se hace el filtrado por `stopwords` conserva las repetidas, ya que se busca medir el largo del abstract.

4. **Capa de Vocabulario Compartido:**

   - Conteo Palabras: acumula frecuencias por idioma, si en ambos idiomas es mayor a 0, cumple la condición de la consulta 3.
   - Top 20: calcula el top 20 final de las frecuencias.

5. **Capa de Promedios:**

   - Acumulación y Filtrado 2021: acumula los artículos de 2021 y posteriores para su procesamiento. Al recibir el promedio necesario, filtra todo lo acumulado.
   - Calculador de Promedio: suma todos los largos y la cantidad total de artículos procesados para obtener el promedio final.

8. Vista de Despliegue

A continuación se detallará el sistema distribuido, qué nodos de cómputo existen y cómo interactúan entre sí. Para ello se mostrará un diagrama de robustez y un diagrama de despliegue.

8.1. Controllers y flujos de datos

Como aclaración antes de pasar al diagrama, hay que decir que todas las colas que se ven en el mismo son queues de RabbitMQ a menos que se indique lo contrario.

El diagrama de robustez que describe el sistema es el siguiente:

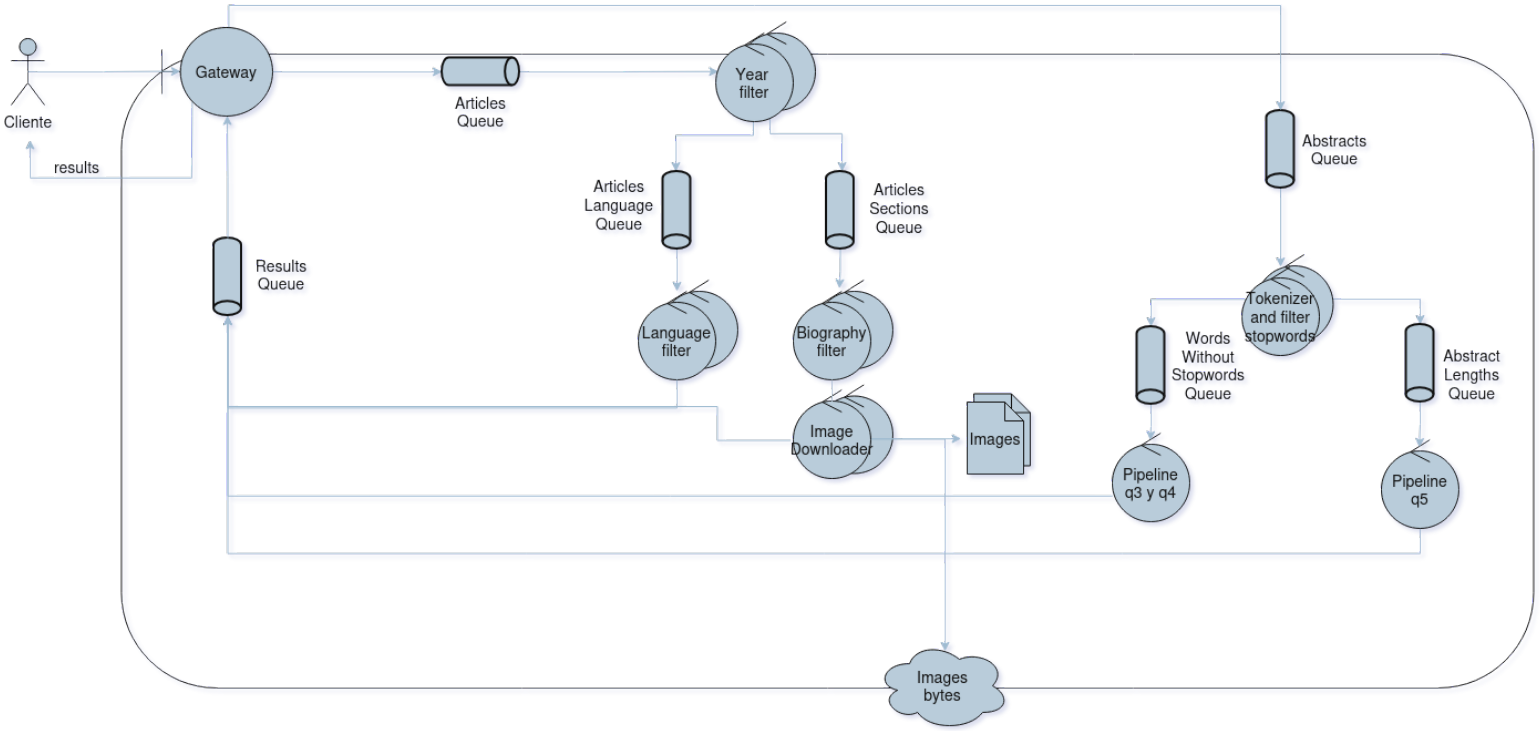


Figura 17: Diagrama de Robustez

En esta primera vista podemos notar cómo el Gateway despacha en 2 colas para el procesamiento de las diferentes consultas.

En el siguiente diagrama vemos el pipeline de procesamiento para las consultas 3 y 4:

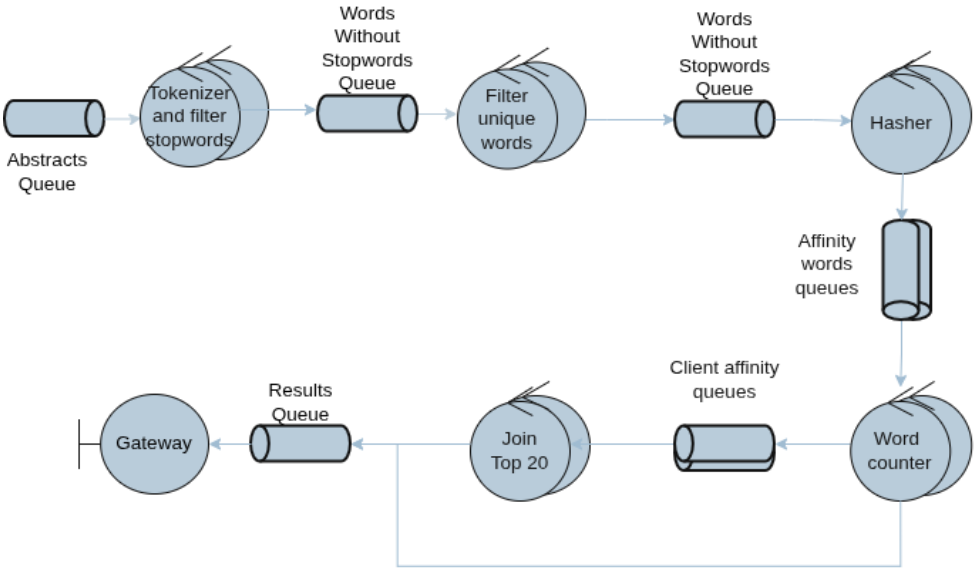


Figura 18: Pipeline Q3 y Q4

En este pipeline podemos ver que disponemos de un hasher para poder enviar las palabras mediante exchanges con affinity por palabra. De esta manera nos aseguramos que la intersección de idiomas para la consulta 3 se pueda realizar en múltiples nodos distribuidos. Luego, se puede ver otro exchange con afinidad por cliente, que, como se explicó en la vista de procesos, no sería un cuello de botella ya que los datos que se reciben de cada cliente en este nodo terminan estando acotados.

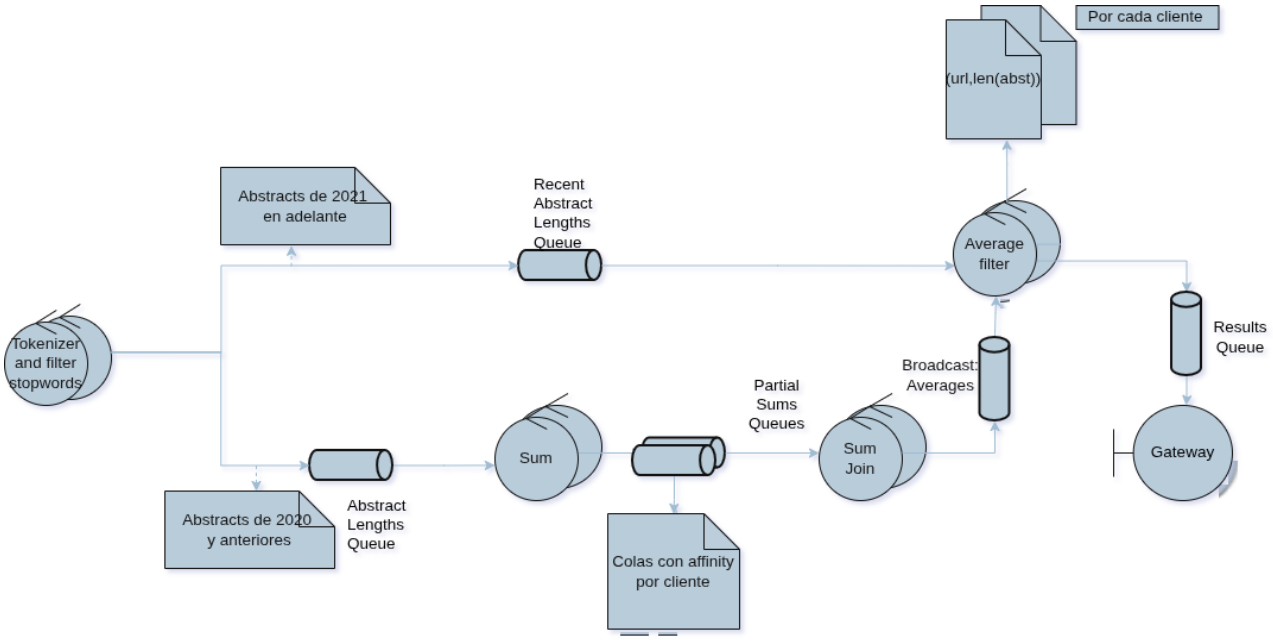


Figura 19: Pipeline Q5

Para la consulta 5 se pueden ver los procesos mencionados anteriormente, los cuales se comunican mediante colas y exchanges. En este caso, se aplica la misma lógica para el affinity por cliente, donde los datos de un solo cliente terminan estando acotados. También, podemos ver que los nodos de filtro,

que tienen que acumular los datos de 2021 en adelante, disponen de archivos locales para el guardado de los datos parciales.

8.2. Nodos y contenedores

El diagrama de despliegue que representa al sistema es el mostrado a continuación:

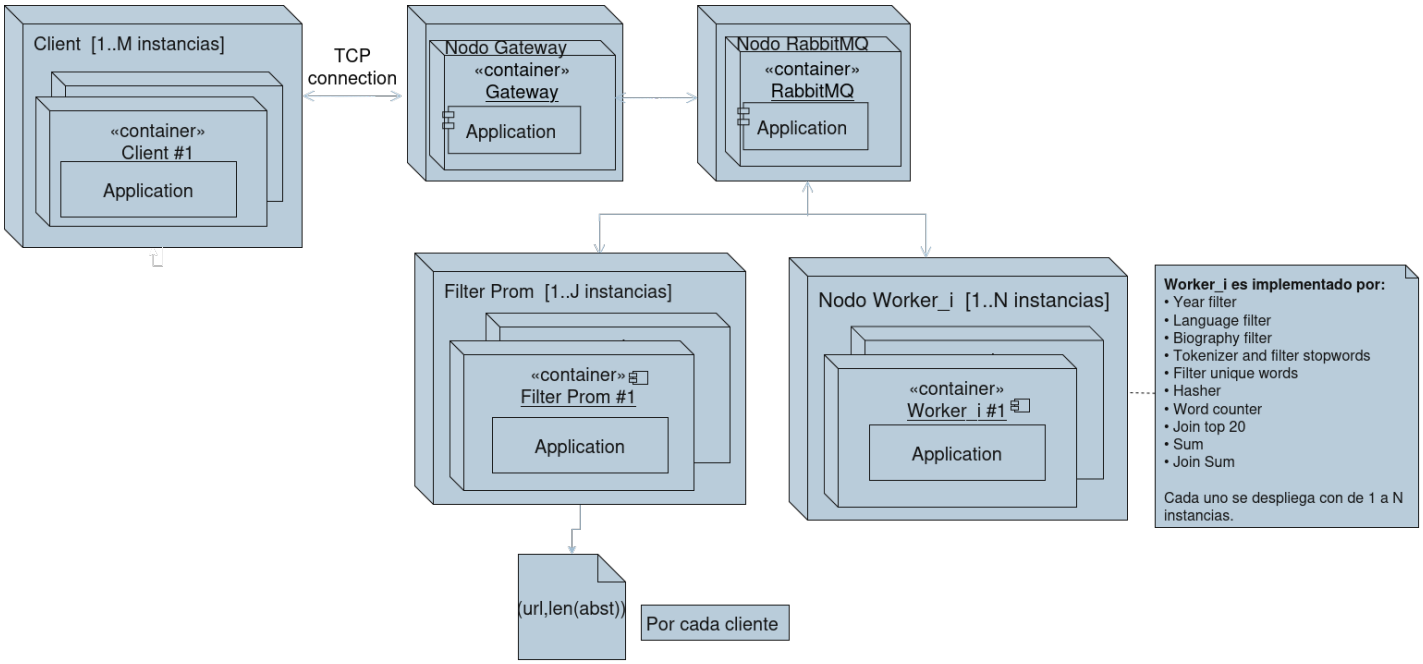


Figura 20: Despliegue

El diagrama de despliegue modela la topología física y de contenedores del sistema, donde múltiples instancias de clientes se comunican mediante conexiones TCP externas únicamente con el Nodo Gateway, el cual actúa como punto de entrada hacia el sistema. Como toda la comunicación dentro del sistema es mediante colas y exchanges de RabbitMQ, todos tienen que estar conectados con el nodo broker. Por otro lado, los clústeres de workers especializados escalables horizontalmente (1..N instancias para cada etapa del pipeline: filtros, tokenizer, hasher, contadores y joins) y hacia los nodos Filter Prom (1..J instancias), los cuales gestionan almacenamiento local en disco con un archivo dedicado por cliente para persistir temporalmente las tuplas (url, len(abst)) hasta la recepción del promedio histórico.

9. Calidad

Esta sección resume los atributos de calidad que guiaron el diseño del sistema y cómo se manifiestan en todas las decisiones descritas en las secciones anteriores.

Escalabilidad

La escalabilidad es la capacidad de un sistema para adaptarse a diferentes ambientes modificando los recursos del sistema. En la práctica, esto significa poder atender más datos o más clientes simultáneos sin rediseñar el sistema. Esto puede lograrse de varias maneras, en este proyecto se buscó que el sistema sea **escalable horizontalmente**, de modo que, ante un cuello de botella, alcance con agregar nodos de la etapa afectada. Para lograrlo se tomaron las siguientes decisiones:

- **Etapas aisladas y comunicadas por colas** (sección 2.1): cada etapa corre en 1..N instancias (Figura 15). Al consumir de una cola, el trabajo se reparte entre las réplicas, de forma tal que si una réplica está saturada deja de tomar mensajes y cede el trabajo a las demás.
- **Reparto por clave**: el Hasher envía cada palabra a un exchange con afinidad por palabra, por lo que el conteo de la consulta 3 se distribuye entre varias instancias de *Word counter*. Los nodos que combinan resultados parciales usan afinidad por cliente, con un estado acotado.
- **Coordinación de EOF independiente de la cantidad de réplicas** (sección 5.3): el protocolo cuenta mensajes y EOF, sin suponer un número fijo de instancias.

Costo y limitaciones: Como mencionamos anteriormente (sección 2.1), más instancias implican más colas y mensajes en RabbitMQ.

Rendimiento

Con la intención de mejorar el rendimiento de la solución, se tomaron las siguientes decisiones:

- **Batching**: la comunicación entre algunos nodos del sistema se realiza mediante envío en *batch*. Esto reduce la cantidad de mensajes en circulación y evita sobrecargar los canales de comunicación.
- **Filtrado temprano**: se busca descartar lo antes posible lo que no sirve. Por ejemplo, el filtro por año descarta lo posterior a 2020 y proyecta solo los campos necesarios, y el Tokenizer quita stopwords antes de calcular las palabras únicas. Esto evita trabajo innecesario en las etapas posteriores y reduce el tráfico por las colas de mensajería.
- **Resultados parciales**: cada etapa entrega los resultados parciales lo antes posible, evitando almacenar información innecesariamente.

Costo y limitaciones: El uso de batching implica que un buffer incompleto no se envía hasta llenarse o hasta que llegue el EOF, lo que puede retrasar resultados parciales de clientes con pocos datos. Por eso, es importante elegir correctamente el tamaño del batch para intentar reducir la latencia lo máximo posible.

Consistencia

Al tener varias instancias es necesario coordinarlas de forma tal de obtener resultados correctos y consistentes con lo esperado por el cliente. Para lograr eso, se propone lo siguiente:

- **Coordinación de EOF** (sección 5.3): una etapa no avisa a la siguiente que terminó hasta asegurarse de haber procesado todos los mensajes del cliente. Para esto se usan dos mecanismos:
  - *Espera los EOF de la etapa anterior*: cada instancia espera recibir el EOF de todas las instancias de la etapa anterior antes de considerar que no llegarán más datos.
  - *Coordinación entre las instancias de una misma etapa*: cada instancia conoce la cantidad total de mensajes enviados por la etapa anterior (`cant_total`) y cuántos procesaron entre todas las instancias (`total_procesados`). Recién cuando ambos valores coinciden, cada instancia envía su EOF a la etapa siguiente.
- **Joins que combinan resultados parciales**: ninguna instancia ve todos los datos de un cliente, por lo que el resultado final se arma en un nodo join.
- **Aislamiento entre clientes**: el `client_id` evita que se mezclen los datos de clientes distintos.

Costo: son barreras de sincronización. La instancia más lenta marca cuándo se puede avanzar.

Mantenibilidad y extensibilidad

La mantenibilidad es la facilidad con la que se puede corregir o modificar un sistema, y la extensibilidad, la facilidad para agregarle funcionalidad nueva con el menor impacto posible sobre lo existente. En este sistema, el lugar donde se espera que haya más cambios es el conjunto de workers: ante una consulta nueva, lo habitual sería agregar o modificar una etapa de procesamiento.

- `BaseWorker` **como plantilla** (sección 6.1.3): La clase abstracta define el esqueleto común a todos los workers: el ciclo de vida (`start` y `stop`), el envío en batch (`emit` y `flush_batch`), la recepción de mensajes de datos y de control, y la cuenta de EOF. Lo único que varía entre workers es el procesamiento en sí, que se delega en el método abstracto `process_data`.
- **Un worker nuevo es una subclase**: para agregar una etapa alcanza con heredar de `BaseWorker` y definir `process_data`, sin tocar el código existente.
- **Sin código repetido**: la lógica de batch, ciclo de vida y sincronización de EOF está escrita una sola vez. Una corrección en cualquiera de ellas se hace en un único lugar y beneficia a todos los workers.
- **Formato de mensaje único**: el subpaquete *Protocol* (sección 6.1.2) centraliza la serialización, por lo que ningún worker conoce los detalles del formato TLV.

Costo: se debe mantener la jerarquía de clases que mantener. Como todos los workers heredan de `BaseWorker`, un cambio en esa clase afecta los afecta a todos, por lo que debe hacerse con cuidado. Además, el comportamiento común puede limitar a los workers que necesiten algo diferente.

Bajo acoplamiento

El acoplamiento es el grado de dependencia entre los componentes de un sistema.Un bajo acoplamiento permite modificar o reemplazar componentes sin afectar al resto del sistema.

- **Sin dependencias cruzadas** (sección 6.1.1): *Client*, *Gateway* y *Workers* se comunican mediante colas y exchanges de RabbitMQ. El paquete *Common* define el formato de los mensajes que comparten.
- `MiddlewareHandler` **oculta RabbitMQ** (sección 6.1.2): los componentes utilizan esta interfaz en lugar de depender directamente de RabbitMQ. Si se cambia de tecnología, solo sería necesario modificar `RabbitMQHandler`, sin modificar los workers, el *Gateway* ni el cliente.

9.1. Apéndice A: Acrónimos y abreviaturas

- **AMQP**: Advanced Message Queuing Protocol, el protocolo que usa RabbitMQ.
- **DAG**: Directed Acyclic Graph, grafo dirigido acíclico.
- **EOF**: End Of File, marca que indica que un proceso terminó de enviar los datos de un cliente.
- **MOM**: Message-Oriented Middleware, middleware orientado a mensajes.
- **NLTK**: Natural Language Toolkit, biblioteca de la que se obtienen las stopwords.
- **Q1 a Q5**: consultas 1 a 5.
- **TCP**: Transmission Control Protocol.
- **TLV**: Type-Length-Value, formato de serialización de los mensajes.
- **UML**: Unified Modeling Language.

9.2. Apéndice B: Definiciones

- **Afinidad (affinity)**: mecanismo por el cual un exchange envía todos los mensajes con una misma clave (una palabra o un cliente) a la misma instancia.
- **Average filter**: worker que guarda en disco los artículos de 2021 en adelante hasta recibir el promedio y luego filtra los que lo superan.

- **Batch**: conjunto de mensajes que se agrupan en un buffer y se envían juntos para reducir el tráfico de red.
- **BaseWorker**: clase abstracta que contiene la lógica común de todos los workers.
- **Broadcast**: envío de un mensaje a todas las instancias de un proceso.
- **Cola (queue)**: estructura de RabbitMQ de la que las instancias de un proceso toman mensajes. Cada mensaje es consumido por una sola instancia.
- **Exchange**: punto de RabbitMQ que distribuye los mensajes a las colas según una regla (afinidad o broadcast).
- **Gateway**: nodo único que recibe los datos de los clientes, les asigna un `client_id`, los envía a los pipelines y devuelve los resultados.
- **Pipeline**: secuencia de procesos conectados por colas que resuelve una o más consultas.
- **Pipes-Filters y Scatter-Gather**: estilos que utiliza el sistema: filtros encadenados para las consultas 1 y 2, y distribución del trabajo seguida de la combinación de resultados parciales para las consultas 3, 4 y 5.
- **Results Queue**: cola por la que llegan todos los resultados al Gateway.
- **Stopword**: palabra trivial (artículos, preposiciones, etc.) que se excluye del conteo.
- **Worker (instancia)**: proceso que implementa una etapa del pipeline. Puede haber varias instancias de cada tipo.